Department of Computer Science & Engineering

Global Campus, Jakkasandra Post, Kanakapura Taluk, Ramanagara District, Pin Code: 562 112

DESIGN AND ANALYSIS OF ALGORITHMS LAB ASSIGNMENT

SUBMITTED TO :

Prof. Katheeja Khan Pathan,
Department of CSE(AI&ML),
JAIN (Deemed-to-be University), Bengaluru.

SUBMITTED BY :-

B. KALYANA SRINIVAS
24BTRCL043
AIML - A

Date : 10-2-26 **LAB ASSIGMENT**

1)

a) **Aim** : Demonstrate Iterative approaches and display the running time of the program.

Source Code :

```
import time
def fib_iterative(n):
    a, b = 0, 1
    for i in range(n):
        print(a, end=" ")
        a, b = b, a + b
n = int(input("Enter a number : "))
start_time = time.time()
print("Fibonacci Series (Iterative):")
fib_iterative(n)
end_time = time.time()
print("\nRunning Time (Iterative):", end_time - start_time, "seconds")
```

Output:

```
Enter a number :  6
Fibonacci Series (Iterative):
0 1 1 2 3 5
Running Time (Iterative): 0.00020647048950195312 seconds
```

b) **Aim:** Demonstrate Recursion and display the running time of the program.

Source Code:

```
import time
def fib_recursive(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib_recursive(n-1) + fib_recursive(n-2)
n = int(input("Enter a number : "))
start_time = time.time()
print("Fibonacci Series (Recursive):")
for i in range(n):
    print(fib_recursive(i), end=" ")
    end_time = time.time()
print("\nRunning Time (Recursive):", end_time - start_time, "seconds")
```

Output:

```
Enter a number :  6
Fibonacci Series (Recursive):
0 1 1 2 3 5
Running Time (Recursive): 0.00022339820861816406 seconds
```

Date : 24-2-26

2)

a) **Aim :** Demonstrate Linear Search in Single and Multi-Dimensional Array.

Linear Search in Single Dimensional Array:

Source Code:

```
def linear_search_1D(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
arr = [10, 25, 30, 45, 50, 65]
key = 45
result = linear_search_1D(arr, key)
if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

Output:

```
Element found at index: 3
```

Linear Search in Multi-Dimensional Array:

Source code:

```
def linear_search_2D(matrix, key):
    for i in range(len(matrix)):
        for j in range(len(matrix[i])):
            if matrix[i][j] == key:
                return i, j
    return -1, -1
matrix = [
    [5, 12, 17],
    [20, 25, 30],
    [35, 40, 45]
]
key = 25
row, col = linear_search_2D(matrix, key)
if row != -1:
    print("Element found at position: Row", row, "Column", col)
else:
    print("Element not found")
```

Output:

```
Element found at position: Row 1 Column 1
```

b) Implement Divide and conquer technique

i) **Aim :** Search an element using Binary Search in sorted and unsorted list.

Source Code :

```
def binary_search(arr, x):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == x:
            return mid
        elif arr[mid] < x:
            low = mid + 1
        else:
            high = mid - 1
    return -1
arr_unsorted = [25, 10, 30, 15, 20]
x = 15
print("Unsorted List:", arr_unsorted)
arr_sorted = sorted(arr_unsorted)
print("Sorted List:", arr_sorted)
result = binary_search(arr_sorted, x)
if result != -1:
    print(f"{x} found at index {result}")
else:
    print("Element not found")
```

Output:

```
Unsorted List: [25, 10, 30, 15, 20]
Sorted List: [10, 15, 20, 25, 30]
15 found at index 1
```

ii) **Aim :** Sort a given sent of elements using the Quick sort method and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted. The elements can be read from a file or can be generated using the random number generator.

Source Code :

```
import random
import time
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    else:
        pivot = arr[0]
        left = [x for x in arr[1:] if x <= pivot]
        right = [x for x in arr[1:] if x > pivot]
        return quick_sort(left) + [pivot] + quick_sort(right)
sizes = [100, 500, 1000, 2000, 5000]
for n in sizes:
    arr = [random.randint(1, 10000) for _ in range(n)]
    start_time = time.time()
    quick_sort(arr)
    end_time = time.time()
    print(f"Time taken to sort {n} elements: {end_time - start_time:.6f} seconds")
```

Output:

```
Time taken to sort 100 elements: 0.000155 seconds
Time taken to sort 500 elements: 0.000659 seconds
Time taken to sort 1000 elements: 0.001526 seconds
Time taken to sort 2000 elements: 0.003115 seconds
Time taken to sort 5000 elements: 0.008164 seconds
```

iii) **Aim :** Sort a given sent of elements using the Merge sort method and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted. The elements can be read from a file or can be generated using the random number generator.

Source Code :

```
import random
import time
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
sizes = [100, 500, 1000, 2000, 5000]
```

```
for n in sizes:
    arr = [random.randint(1, 10000) for _ in range(n)]
    start_time = time.time()
    merge_sort(arr)
    end_time = time.time()
    print(f"Time taken to sort {n} elements: {end_time - start_time:.6f} seconds")
```

Output:

```
Time taken to sort 100 elements: 0.000179 seconds
Time taken to sort 500 elements: 0.001206 seconds
Time taken to sort 1000 elements: 0.001544 seconds
Time taken to sort 2000 elements: 0.005365 seconds
Time taken to sort 5000 elements: 0.014006 seconds
```

c) **Aim :** Print all the nodes reachable from a given starting node in a digraph using BFS and DFS method.

BFS (Breadth First Search)

Source Code :

```
from collections import deque
graph = {
    0: [1, 2],
    1: [3],
    2: [3, 4],
    3: [],
    4: []
}

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node, end=" ")
```

```
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                queue.append(neighbor)

start_node = 0
print("BFS Traversal:")
bfs(graph, start_node)
```

Output :

```
BFS Traversal:
0 1 2 3 4
```

DFS (Depth First Search)

Source Code :

```
graph = {
    0: [1, 2],
    1: [3],
    2: [3, 4],
    3: [],
    4: []
}
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    print(start, end=" ")
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
start_node = 0
print("DFS Traversal:")
dfs(graph, start_node)
```

Output :

```
DFS Traversal:
0 1 3 2 4
```

Date : 03-03-26

3) a)

Aim : Demonstrate Minimum Cost Spanning Trees.

Minimum Spanning Tree using Prim's Algorithm

Source Code:

```
import sys
graph = [
    [0, 2, 0, 6, 0],
    [2, 0, 3, 8, 5],
    [0, 3, 0, 0, 7],
    [6, 8, 0, 0, 9],
    [0, 5, 7, 9, 0]
]
V = 5
selected = [False] * V
selected[0] = True
print("Prim's Minimum Spanning Tree")
print("Edge : Weight")
for _ in range(V - 1):
    minimum = sys.maxsize
    x = 0
    y = 0
    for i in range(V):
        if selected[i]:
```

```
        for j in range(V):
            if (not selected[j]) and graph[i][j]:
                if minimum > graph[i][j]:
                    minimum = graph[i][j]
                    x = i
                    y = j

    print(x, "-", y, ":", graph[x][y])
    selected[y] = True
```

Output:

```
Prim's Minimum Spanning Tree
Edge : Weight
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6
```

Minimum Spanning Tree using Kruskal's Algorithm

Source Code:

```
edges = [
    (0, 1, 10),
    (0, 2, 6),
    (0, 3, 5),
    (1, 3, 15),
    (2, 3, 4)
]
edges.sort(key=lambda x: x[2])
parent = [0, 1, 2, 3]
def find(i):
    if parent[i] == i:
        return i
    return find(parent[i])
def union(x, y):
```

```
    parent[x] = y
print("\nKruskal's Minimum Spanning Tree")
print("Edge : Weight")
for u, v, w in edges:
    x = find(u)
    y = find(v)

    if x != y:
        print(u, "-", v, ":", w)
        union(x, y)
```

Output:

```
Kruskal's Minimum Spanning Tree
Edge : Weight
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
```

b)

Aim: Implement Single Source Shortest Paths algorithm.

Single Source Shortest Path using Dijkstra's Algorithm

Source Code:

```
import sys
graph = [
    [0, 10, 0, 30, 100],
    [10, 0, 50, 0, 0],
    [0, 50, 0, 20, 10],
    [30, 0, 20, 0, 60],
    [100, 0, 10, 60, 0]
]
V = 5
```

```
def dijkstra(graph, source):
    dist = [sys.maxsize] * V
    dist[source] = 0
    visited = [False] * V
    for _ in range(V):
        min_dist = sys.maxsize
        for v in range(V):
            if not visited[v] and dist[v] < min_dist:
                min_dist = dist[v]
                u = v
        visited[u] = True
        for v in range(V):
            if graph[u][v] > 0 and not visited[v] and dist[v] > dist[u] + graph[u][v]:
                dist[v] = dist[u] + graph[u][v]

    print("Vertex \t Distance from Source")
    for i in range(V):
        print(i, "\t", dist[i])
dijkstra(graph, 0)
```

Output:

```
Vertex   Distance from Source
0        0
1        10
2        50
3        30
4        60
```

c)

Aim: Implement All-Pairs Shortesh Path Problem using Floyd's algorithm.

All-Pairs Shortest Path using Floyd's Algorithm

Source Code:

```
INF = 999
graph = [
    [0, 3, INF, 7],
    [8, 0, 2, INF],
    [5, INF, 0, 1],
    [2, INF, INF, 0]
]
V = 4
def floyd_warshall(graph):
    dist = graph.copy()
    for k in range(V):
        for i in range(V):
            for j in range(V):
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
    print("Shortest distance matrix:")
    for i in range(V):
        for j in range(V):
            print(dist[i][j], end=" ")
        print()
floyd_warshall(graph)
```

<u>Output</u>:

```
Shortest distance matrix:
0 3 5 6
5 0 2 3
3 6 0 1
2 5 7 0
```

d) Demonstrate Backtracking technique

i)

Aim : n-Queens problem (n = 4, n =8)

For n = 4

Source Code:

```
def is_safe(board, row, col, n):
    for i in range(row):
        if board[i] == col or \
            board[i] - i == col - row or \
            board[i] + i == col + row:
             return False
    return True
def solve(board, row, n):
    if row == n:
        print(board)
        return
    for col in range(n):
        if is_safe(board, row, col, n):
            board[row] = col
            solve(board, row + 1, n)
n = 4
board = [-1] * n
print("Solutions for 4-Queens:")
solve(board, 0, n)
```

Output:

```
Solutions for 4-Queens:
[1, 3, 0, 2]
[2, 0, 3, 1]
```

For n = 8

Source Code:

```
def is_safe(board, row, col, n):
    for i in range(row):
        if board[i] == col or \
           board[i] - i == col - row or \
           board[i] + i == col + row:
            return False
    return True
def solve(board, row, n):
    if row == n:
        print(board)
        return
    for col in range(n):
        if is_safe(board, row, col, n):
            board[row] = col
            solve(board, row + 1, n)
n = 8
board = [-1] * n
print("Solutions for 8-Queens:")
solve(board, 0, n)
```

Output:

```
Solutions for 8-Queens:
[0, 4, 7, 5, 2, 6, 1, 3]
[0, 5, 7, 2, 6, 3, 1, 4]
[0, 6, 3, 5, 7, 1, 4, 2]
[0, 6, 4, 7, 1, 3, 5, 2]
[1, 3, 5, 7, 2, 0, 6, 4]
[1, 4, 6, 0, 2, 7, 5, 3]
[1, 4, 6, 3, 0, 7, 5, 2]
[1, 5, 0, 6, 3, 7, 2, 4]
[1, 5, 7, 2, 0, 3, 6, 4]
[1, 6, 2, 5, 7, 4, 0, 3]
[1, 6, 4, 7, 0, 3, 5, 2]
[1, 7, 5, 0, 2, 4, 6, 3]
[2, 0, 6, 4, 7, 1, 3, 5]
[2, 4, 1, 7, 0, 6, 3, 5]
[2, 4, 1, 7, 5, 3, 6, 0]
[2, 4, 6, 0, 3, 1, 7, 5]
[2, 4, 7, 3, 0, 6, 1, 5]
[2, 5, 1, 4, 7, 0, 6, 3]
[2, 5, 1, 6, 0, 3, 7, 4]
[2, 5, 1, 6, 4, 0, 7, 3]
[2, 5, 3, 0, 7, 4, 6, 1]
[2, 5, 3, 1, 7, 4, 6, 0]
[2, 5, 7, 0, 3, 6, 4, 1]
[2, 5, 7, 0, 4, 6, 1, 3]
[2, 5, 7, 1, 3, 0, 6, 4]
[2, 6, 1, 7, 4, 0, 3, 5]
```

ii)

Aim : Solving Sudoku puzzle.

Source Code:

```
def is_safe(board, row, col, num):
    # Check row
    for x in range(9):
        if board[row][x] == num:
            return False
    # Check column
    for x in range(9):
        if board[x][col] == num:
            return False
    # Check 3x3 box
    startRow = row - row % 3
    startCol = col - col % 3
    for i in range(3):
        for j in range(3):
            if board[i + startRow][j + startCol] == num:
                return False
    return True
def solve_sudoku(board):
    for row in range(9):
        for col in range(9):
            if board[row][col] == 0:
                for num in range(1, 10):
                    if is_safe(board, row, col, num):
                        board[row][col] = num
                        if solve_sudoku(board):
                            return True
                        board[row][col] = 0
                return False
```

```
    return True

board = [
[3,0,6,5,0,8,4,0,0],
[5,2,0,0,0,0,0,0,0],
[0,8,7,0,0,0,0,3,1],
[0,0,3,0,1,0,0,8,0],
[9,0,0,8,6,3,0,0,5],
[0,5,0,0,9,0,6,0,0],
[1,3,0,0,0,0,2,5,0],
[0,0,0,0,0,0,0,7,4],
[0,0,5,2,0,6,3,0,0]
]
solve_sudoku(board)
print("Solved Sudoku:")
for row in board:
    print(row)
```

Output:

```
Solved Sudoku:
[3, 1, 6, 5, 7, 8, 4, 9, 2]
[5, 2, 9, 1, 3, 4, 7, 6, 8]
[4, 8, 7, 6, 2, 9, 5, 3, 1]
[2, 6, 3, 4, 1, 5, 9, 8, 7]
[9, 7, 4, 8, 6, 3, 1, 2, 5]
[8, 5, 1, 7, 9, 2, 6, 4, 3]
[1, 3, 8, 9, 4, 7, 2, 5, 6]
[6, 9, 2, 3, 5, 1, 8, 7, 4]
[7, 4, 5, 2, 8, 6, 3, 1, 9]
```

Date : 24-03-26

4) a)

Aim : Demonstrate 0/1 knapsack problem using Least Cost Branch and Bound.

Source Code:

```
from queue import Queue
class Node:
    def __init__(self, level, profit, weight, bound):
        self.level = level
        self.profit = profit
        self.weight = weight
        self.bound = bound
def bound(node, n, W, p, w):
    if node.weight >= W:
        return 0
    profit = node.profit
    i = node.level + 1
    tot_w = node.weight
    while i < n and tot_w + w[i] <= W:
        tot_w += w[i]
        profit += p[i]
        i += 1
    if i < n:
        profit += (W - tot_w) * p[i] / w[i]
    return profit
def knapsack(W, w, p, n):
    Q = Queue()
    maxProfit = 0
    Q.put(Node(-1, 0, 0, 0))

    while not Q.empty():
```

```
        u = Q.get()

        if u.level == n - 1:
            continue

        v = Node(u.level + 1,
                 u.profit + p[u.level + 1],
                 u.weight + w[u.level + 1], 0)
        if v.weight <= W and v.profit > maxProfit:
            maxProfit = v.profit
        v.bound = bound(v, n, W, p, w)
        if v.bound > maxProfit:
            Q.put(v)
        v = Node(u.level + 1, u.profit, u.weight, 0)
        v.bound = bound(v, n, W, p, w)
        if v.bound > maxProfit:
            Q.put(v)
    return maxProfit
profit = [60, 100, 120]
weight = [10, 20, 30]
W = 50
print("Max Profit =", knapsack(W, weight, profit, len(profit)))
```

Output:

```
Max Profit = 220
```

b) **Aim :** Demonstrate Travelling salesperson problem using Branch and Bound.

Source Code:

```
import sys
N = 4
cost = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
visited = [False] * N
min_cost = sys.maxsize
def tsp(curr_pos, count, curr_cost):
    global min_cost
    if count == N and cost[curr_pos][0]:
        min_cost = min(min_cost, curr_cost + cost[curr_pos][0])
        return
    for i in range(N):
        if not visited[i] and cost[curr_pos][i]:
            visited[i] = True
            tsp(i, count + 1, curr_cost + cost[curr_pos][i])
            visited[i] = False
visited[0] = True
tsp(0, 1, 0)
print("Minimum travelling cost =", min_cost)
```

Output:

```
Minimum travelling cost = 80
```